Py_DAW

SQLite Datenbank Handbuch

Version 0.0.20.897 — ChronoScaleStudio

Vollständige Referenz für die integrierte SQLite-Datenbank

Alle Parameter sind änderbar — nichts ist hardcoded

1. Einführung

Py_DAW speichert alle Einstellungen, Parameter, Presets und Projekt-Metadaten in einer SQLite-Datenbank. Die Datenbank ist das Rückgrat der DAW — sie merkt sich deine Konfiguration, Plugin-States, MIDI-Mappings und UI-Einstellungen über Sessions hinweg.

Wichtig: Alle Werte in der Datenbank sind reine Defaults. Du kannst jeden einzelnen Wert jederzeit ändern — über die DAW-Oberfläche, die integrierte SQLite Console oder die Kommandozeile. Nichts ist hardcoded.

1.1 Wo liegt die Datenbank?

Die DB-Dateien liegen im Konfigurations-Verzeichnis:

```
~/.config/ChronoScaleStudio/
```

Dort findest du drei Dateien:

Datei	Inhalt	Tabellen
pydaw.db	Haupt-Datenbank (alle Einstellungen)	44
preset_database.db	Presets mit Volltext-Suche (FTS5)	2
param_registry.db	Parameter-Registry (Instrumente)	1

1.2 DB-Status prüfen

Du hast drei Möglichkeiten den Status zu prüfen:

In der DAW: SQLite-Badge

Unten rechts in der Status-Bar siehst du das SQLite-Badge (blau-teal 'Sql' Symbol). Klick darauf öffnet den DB-Status-Dialog mit allen Tabellen und Zeilenanzahlen.

Rechtsklick auf das Badge zeigt ein Kontextmenü:

Aktion	Beschreibung
📊 DB-Status anzeigen	Öffnet den Health-Dialog mit allen Tabellen
🗄️ SQLite Console öffnen	Öffnet den integrierten DB-Editor
📁 DB-Pfad kopieren	Kopiert den Dateipfad in die Zwischenablage

Kommandozeile: Health Check

```
python3 db_health_check.py
python3 db_health_check.py --detail    # Mit Beispiel-Daten
python3 db_health_check.py --path      # Nur DB-Pfad ausgeben
```

Direkt mit sqlite3

```
sqlite3 ~/.config/ChronoScaleStudio/pydaw.db
sqlite> .tables
sqlite> SELECT * FROM settings LIMIT 10;
```

2. SQLite Console — Der integrierte DB-Editor

Die SQLite Console ist ein vollwertiger Datenbank-Editor direkt in der DAW — genau wie die Python Console, aber für die Datenbank.

2.1 Console öffnen

Die SQLite Console erreichst du über:

```
Menü → Projekt → 🗄 SQLite Console...
Tastenkürzel: Ctrl+Alt+S
```

Die Console erscheint als Dock-Widget am unteren Bildschirmrand. Du kannst sie frei verschieben, andocken oder abreißen.

2.2 Aufbau der Console

Die Console besteht aus drei Bereichen:

Bereich	Position	Funktion
Tabellen-Liste	Links	Alle Tabellen mit Zeilenanzahl — Klick zeigt Daten
Daten-Grid	Rechts oben	Tabelleninhalt — Doppelklick zum Editieren
SQL Console	Rechts unten	Beliebiges SQL eingeben und ausführen

2.3 Werte anzeigen und ändern

So änderst du einen Wert in der Datenbank:

1. Klicke links auf eine Tabelle (z.B. 'settings')
2. Die Daten erscheinen rechts im Grid
3. Doppelklicke auf die Zelle die du ändern willst
4. Tippe den neuen Wert ein
5. Drücke Enter — der Wert wird sofort gespeichert

💡 *Die Zelle wird kurz grün wenn das Update erfolgreich war, rot bei einem Fehler.*

2.4 SQL direkt eingeben

Im SQL-Eingabefeld kannst du beliebiges SQL ausführen:

Daten lesen

```
SELECT * FROM settings WHERE key LIKE '%audio%'
SELECT name, bpm, track_count FROM project_meta
SELECT * FROM themes WHERE name = 'Dark'
```

Daten ändern

```
UPDATE settings SET value = '48000' WHERE key = 'sample_rate'
UPDATE audio_config SET value = '256' WHERE key = 'buffer_size'
UPDATE themes SET active = 1 WHERE name = 'Midnight Blue'
```

> ⚠️ **DROP, ALTER und TRUNCATE zeigen einen Sicherheits-Dialog bevor sie ausgeführt werden.**

Daten einfügen

```
INSERT INTO keybindings (shortcut, action) VALUES ('Ctrl+M', 'toggle_metronome')
```

Daten löschen

```
DELETE FROM browser_recents WHERE file_type = 'project'
```

2.5 Datenbank wechseln

Oben in der Console gibt es ein Dropdown um zwischen den drei Datenbanken zu wechseln:

- pydaw.db — Haupt-Datenbank (Einstellungen, Themes, Plugins, Projekte)
- preset_database.db — Alle Presets mit Volltext-Suche
- param_registry.db — Parameter-Definitionen für Instrumente

3. Tabellen-Referenz — Haupt-Datenbank (pydaw.db)

Die Haupt-Datenbank enthält 44 Tabellen, aufgeteilt in logische Gruppen.

3.1 Einstellungen & Konfiguration

Tabelle	Inhalt	Typische Werte
settings	Allgemeine App-Einstellungen (Key-Value)	last_project, theme, language
audio_config	Audio-Device-Einstellungen	sample_rate, buffer_size, backend
recording_config	Aufnahme-Einstellungen	count_in_bars, format, channels
export_config	Export-Einstellungen	format, bit_depth, normalize
input_config	MIDI/Audio Input-Routing	input_device, channel_filter
ai_config	KI-Assistent Einstellungen	api_key, model, temperature
accessibility_config	Barrierefreiheit	large_text, reduce_motion, focus_ring

3.2 Oberfläche & Themes

Tabelle	Inhalt	Änderbar über
themes	Farbschemata (30+ Farben pro Theme)	Ansicht → Theme wählen
keybindings	Tastaturkürzel	Einstellungen → Shortcuts
ui_layout	UI-Layout Einstellungen	Dock-Positionen, Panel-Größen
browser_places	Browser-Lesezeichen (Orte)	Browser → Orte verwalten
browser_recents	Zuletzt geöffnete Dateien	Automatisch beim Öffnen

3.3 Plugins & Instrumente

Tabelle	Inhalt	Wann befüllt
plugin_cache	Gescannte VST3/CLAP/LV2 Plugins	Beim Plugin-Scan
instruments	Registrierte Instrumente (7 Stück)	Beim Start
factory_samples	Factory Sample-Definitionen (30)	Beim Start
plugin_states	Plugin-State Crash Recovery	Bei jedem Parameter-Änderung
plugin_crash_log	Plugin-Absturzprotokoll	Wenn ein Plugin crasht

3.4 MIDI & Controller

Tabelle	Inhalt	Änderbar über
midi_mappings	CC-Zuordnungen (Controller → Parameter)	MIDI Learn oder hier

controller_profiles	Hardware-Profile (Launchpad, APC, etc.)	Automatisch erkannt
groove_templates	12 Groove-Templates (Swing, MPC, etc.)	Hier oder Groove-Editor
scales	42 Tonleitern	Hier oder Scale-Lock

3.5 Projekt-Mirror (Crash Recovery)

Diese Tabellen spiegeln den Projekt-Zustand nach jedem Speichern. Die JSON-Projektdatei bleibt die autoritative Quelle — SQLite dient als Backup.

Tabelle	Inhalt	Zeilen pro Projekt
project_meta	BPM, Sample-Rate, Taktart, Punch, etc.	1
project_tracks	Alle Tracks (Volume, Pan, Routing, FX)	So viele wie Tracks
project_clips	Alle Clips (Position, Länge, Gain, etc.)	So viele wie Clips
project_midi_notes	MIDI-Noten einzeln	Hunderte bis Tausende
project_automation	Automation Lanes + Breakpoints	Pro automatisiertem Parameter

3.6 History & Protokoll

Tabelle	Inhalt	Automatisch befüllt
undo_history	Undo-Snapshots (Crash Recovery)	Bei jeder Änderung
macro_recordings	Gespeicherte Macro-Aufnahmen	Macro → Stop
script_history	Python Console Befehlshistorie	Bei jedem Befehl
collab_sessions	Kollaborations-Sitzungen	Bei Collab-Disconnect
git_history	Git-Commits (wenn Git aktiv)	Bei jedem Commit
recent_projects	Zuletzt geöffnete Projekte	Bei Projekt-Öffnen
recent_media	Zuletzt importierte Dateien	Bei File-Import
export_history	Export-Protokoll	Bei jedem Export

3.7 UI-State (pro Projekt)

Tabelle	Inhalt	Wann gespeichert
arranger_state	Zoom, Scroll, Track-Höhe im Arranger	Bei Ctrl+S
pianoroll_state	Zoom, Scroll, Note-Höhe im Piano Roll	Bei Ctrl+S
notation_state	Notation-Editor Einstellungen	Bei Ctrl+S
video_state	Video-Track Konfiguration + SMPTE	Bei Video-Laden
podcast_state	Podcast-Modus Einstellungen	Bei EQ-Preset-Wechsel

3.8 Weitere

Tabelle	Inhalt
fx_chain_presets	Gespeicherte FX-Chain Presets

engine_state	Audio-Engine Konfiguration (Key-Value)
note_expression_index	Index für MPE-Expressionsdaten
cloud_sync_log	Cloud-Synchronisations-Protokoll
cloud_conflicts	Cloud-Konflikte (lokal vs. remote)
mastering_presets	Mastering-Chain Presets (LUFS Targets)
undo_command_meta	Undo-Kommando Metadaten

4. Preset-Datenbank (preset_database.db)

Die Preset-Datenbank speichert alle Presets mit Volltext-Suche (FTS5). Du kannst nach Name, Tags, Beschreibung und Charakter suchen.

Tabelle	Inhalt
presets	Alle Presets (Factory + User + Community + KI)
presets_fts	FTS5 Volltext-Index für schnelle Suche

4.1 Preset durchsuchen

```
SELECT name, instrument, category, tags FROM presets WHERE instrument = 'aeterna'
SELECT * FROM presets WHERE name LIKE '%pad%' AND category = 'Pad'
```

4.2 Preset-Volltext-Suche

```
SELECT * FROM presets_fts WHERE presets_fts MATCH 'warm analog reverb'
```

💡 *FTS5 ist extrem schnell — auch bei tausenden Presets < 1ms Antwortzeit.*

4.3 Favoriten verwalten

```
UPDATE presets SET is_favorite = 1 WHERE name = 'Warm Analog Pad'
SELECT name, instrument FROM presets WHERE is_favorite = 1
```

5. Praxisbeispiele

5.1 Audio Buffer-Size ändern

Du willst die Buffer-Size auf 128 Samples setzen:

```
UPDATE audio_config SET value = '128' WHERE key = 'buffer_size'
```

💡 *Änderungen an Audio-Config werden beim nächsten Engine-Neustart wirksam.*

5.2 Tastaturkürzel anpassen

```
-- Aktuelles Kürzel anzeigen
SELECT * FROM keybindings WHERE action LIKE '%metronome%'
-- Neues Kürzel setzen
UPDATE keybindings SET shortcut = 'M' WHERE action = 'toggle_metronome'
```

5.3 Theme-Farben anpassen

```
-- Alle Farben des Dark Themes anzeigen
SELECT * FROM themes WHERE name = 'Dark'
-- Hintergrundfarbe ändern
UPDATE themes SET bg_primary = '#1a1d23' WHERE name = 'Dark'
```

5.4 MIDI-Mapping manuell erstellen

```
INSERT INTO midi_mappings (cc_number, param, track_id, channel)
VALUES (74, 'filter_cutoff', 'trk_abc123', 0)
```

5.5 Problematische Plugins finden

```
SELECT plugin_id, plugin_name, COUNT(*) as crashes
FROM plugin_crash_log
GROUP BY plugin_id ORDER BY crashes DESC
```

5.6 Projekt-Übersicht aus der DB

```
SELECT project_name, bpm, track_count, clip_count,
       datetime(synced_at, 'unixepoch', 'localtime') as letztes_save
FROM project_meta ORDER BY synced_at DESC
```

5.7 Export-Historie anzeigen

```
SELECT output_path, format, sample_rate, bit_depth,
       datetime(exported_at, 'unixepoch', 'localtime') as datum
FROM export_history ORDER BY exported_at DESC LIMIT 10
```

5.8 Groove-Template bearbeiten

```
-- Alle Grooves anzeigen
SELECT name, category, swing_amount FROM groove_templates
-- Swing-Wert anpassen
UPDATE groove_templates SET swing_amount = 65.0 WHERE name = 'Swing 60% (Medium)'
```

6. Architektur & Design-Prinzipien

6.1 DB-first mit Fallback

Alle Reads folgen dem gleichen Pattern:

```
# 1. Versuche aus der DB zu lesen
db_result = try_read_from_db()
if db_result:
    return db_result
# 2. Fallback auf altes System (JSON, QSettings, In-Memory)
return old_source()
```

💡 *Wenn die DB fehlt oder leer ist, funktioniert die DAW ganz normal mit Defaults.*

6.2 Lazy-Init Singletons

Jedes DB-Modul ist ein Singleton das sich erst beim ersten Zugriff initialisiert. Kein Startup-Overhead wenn du die DB nicht brauchst.

6.3 WAL-Modus

Alle Datenbanken nutzen den SQLite WAL (Write-Ahead Logging) Modus. Das bedeutet:

- Lesen und Schreiben können gleichzeitig stattfinden
- Kein Blocking der GUI beim DB-Zugriff
- Crash-sicher — keine halben Schreibvorgänge

6.4 JSON bleibt Primary

Die Projektdatei (.pydaw.json) bleibt die autoritative Quelle für Projekt-Daten. Die SQLite-Tabellen project_meta, project_tracks, project_clips etc. sind nur ein Mirror für Crash Recovery und schnelle Queries. Du verlierst nichts wenn du die DB löschst.

6.5 Die 7 DB-Module

Modul	Phasen	Verantwortlich für
db_manager.py	Basis	Singleton-Verbindung, WAL, Backup
app_database.py	P19-26	Keys, UI, Audio/Recording/Export Config
project_state_db.py	P5,10,12,13	Projekt-Mirror (Tracks/Clips/MIDI)
extended_state_db.py	P11,14,16-18,23-24,29-30	Undo, Macros, Collab, Git, FX, Engine
final_state_db.py	P27,31-33,39-42	Cloud, UI-State, Video, Podcast, Mastering
fileio_state_db.py	P35-36	Recent Files, Export History, Crash Log
plugin_host_state_db.py	P28	Plugin State Cache (Crash Recovery)

7. Sicherheit & Backup

7.1 DB sichern

```
cp ~/.config/ChronoScaleStudio/pydaw.db ~/backup_pydaw.db
```

💡 *Die DB ist eine einzelne Datei — einfach kopieren reicht.*

7.2 DB zurücksetzen

Wenn du komplett von vorne anfangen willst:

```
rm ~/.config/ChronoScaleStudio/pydaw.db
```

Beim nächsten Start der DAW werden alle Tabellen mit Factory-Defaults neu angelegt.

⚠️ **Das löscht alle deine Einstellungen, MIDI-Mappings und Themes. Projekte (JSON-Dateien) sind davon nicht betroffen.**

7.3 Was passiert bei Korruption?

SQLite ist extrem robust. Falls die DB trotzdem korrupt wird:

1. Die DAW startet trotzdem (Fallback auf Defaults)
2. Du siehst eine Warnung im Log
3. Lösche die DB und starte neu — alles wird frisch angelegt

8. Quick Reference Card

Die wichtigsten Befehle auf einen Blick:

Console öffnen

```
Ctrl+Alt+S                          # In der DAW
sqlite3 ~/.config/ChronoScaleStudio/pydaw.db   # Terminal
```

Tabellen anzeigen

```
SELECT name FROM sqlite_master WHERE type='table' ORDER BY name;
```

Einstellungen lesen

```
SELECT * FROM settings;
SELECT * FROM audio_config;
SELECT * FROM themes WHERE name = 'Dark';
```

Einstellungen ändern

```
UPDATE settings SET value = 'Dark' WHERE key = 'theme';
UPDATE audio_config SET value = '256' WHERE key = 'buffer_size';
```

Plugin-Infos

```
SELECT * FROM plugin_cache WHERE format = 'vst3';
SELECT plugin_id, COUNT(*) FROM plugin_crash_log GROUP BY plugin_id;
```

Presets suchen

```
SELECT name, category FROM presets WHERE instrument = 'aeterna';
SELECT * FROM presets_fts WHERE presets_fts MATCH 'warm pad';
```

Projekt-Info

```
SELECT project_name, bpm, track_count FROM project_meta;
SELECT name, kind, volume FROM project_tracks WHERE project_path LIKE '%MeinSong%';
```

Aufräumen

```
DELETE FROM export_history WHERE exported_at < strftime('%s', 'now', '-30 days');
DELETE FROM plugin_crash_log WHERE crashed_at < strftime('%s', 'now', '-90 days');
```